Metaheuristics

- **Meta**- Greek word for upper level methods
- **Heuristics** – Greek word heuriskein – art of discovering new strategies to solve problems.

Exact methods	Approximate methods
Linear Programming, Integer Programming, Non Linear Programming , Dynamic Programming Branch and Bound	Heuristics Metaheuristics

Metaheuristics

- **Combinatorial Optimization problems** – a general class of IP problems with discrete decision variables and finite solution space. Objective function and constraints could be non-linear too. Uses relaxation techniques to prune the search space.
- **Constraint Programming problems** – used for timetabling and scheduling problems. Uses constraint propagation techniques that reduces the variable domain. Declaration of variables is a lot more compact.

Need for Metaheuristics

- What if the objective function **can only be simulated** and there is no (or an inaccurate) mathematical model that connect the variables
- **Large solution space**
  Ex 1: TSP, 5 cities have 120 solutions (5!), 10 cities have 10! 3.6 million, 75 cities have 2.5×10^{109} solutions
  Ex 2: Parallel machine job scheduling: process n jobs on m identical machines. There are m^n solutions
- **Complexity** - time and space complexity Time is the number of steps required to solve a problem of size n. We are looking for the worst-case asymptotic bound on the step count (not an exact count). Asymptotic behavior is the limiting behavior as n tends to a large number.

Metaheuristics

- **The idea**: search the solution space directly. No math models, only a set of algorithmic steps, iterative method.
- **Trajectory based methods** (single solution based methods)
  - Search process is characterized by a trajectory in the search space
  - It can be viewed as an evolution of a solution in (discrete) time of a dynamical system
    - Tabu Search, Simulated Annealing, Iterated Local search, variable neighborhood search, guided local search
- **Population based methods**
  - Every step of the search process has a population – a set of- solutions
  - It can be viewed as an evolution of a set of solutions in (discrete) time of a dynamical system
    - Genetic algorithms, Swarm intelligence - Ant colony optimization, Bee colony optimization
- Hybrid methods
- Parallel metaheuristics: parallel and distributed computing- independent and cooperative search

Metaheuristics

- **Diversification** and **intensification** of the search are the two strategies for search in Metaheuristics. One must strike a balance between them. Too much of either one will yield poor solutions. Remember that you have only a limited amount of time to search and you are also looking for a good quality solution. Quality vs Time tradeoff.
  - Trajectory based methods are heavy on intensification,while population based methods are heavy on diversification.
- **Diversification** means to generate diverse solutions so as to explore the search space on the global scale,while **intensification** means to focus on the search in a local region by exploiting the information that a current good solution is found in this region.

Classification of metaheuristics

- **Nature inspired** (swarm intelligence from biology) vs nonnature inspired (simulated annealing from physics)
- **Memory usage** (tabu search) vs **memoryless** methods (local search, simulated annealing SA) **Deterministic** (tabu, local search) vs **stochastic** (GA, SA) metaheuristics
  - Deterministic – same initial solution will lead to same final solution after several search steps
  - Stochastic – same initial solution will lead to different final solutions due to some random rules in the algorithm
- **Population based** (manipulates a whole population of solutions – exploration/diversification) vs single solution based search (manipulates a **single solution** – exploitation/intensification).
- **Iterative vs greedy**. **Iterative** – start with a complete solution(s) and transform at each iteration. **Greedy**- start with an empty solution and add decision variables till a complete solution is obtained.

Classification of metaheuristics

Diversification	Instancification
Genetic Algorithm Ant Colony Optimization Particle Swarm Optimization ...	Simulated Annealing Tabu Search Variable Neighborhood Search

When to use Metaheuristics

- If one can use exact methods then do not use Metaheuristics.
- P class problem with large number of solutions. P-time algorithms are known but too expensive to implement
- Dynamic optimization- real-time optimization- Metaheuristics can reduce search time and we are looking for good enough solutions.
- A difficult NP-hard problem - even a moderate size problem.

Evaluation of Results

- Best Metaheuristic approach- does not exist for any problem. No proof of optimality exists and you don't care for one either.
- A heuristic approach designed for problem A does not always apply to problem B. It is context-dependent.
- Questions: Is the metaheuristic algorithm well designed and does it behave well? No common agreement exist in the literature.
- What do we look for during implementation
  - Easy to apply for hard problems
  - Intensify: should be able to find a local optima
  - Diversify: should be able to widely explore the solution space
  - A greedy solution is a good starting point for an iterative search

Evaluation of results

Evaluation criteria include:

- **Ability to find good/optimal solutions**. No way to prove optimality unless all solutions are exhaustively enumerated, which is infeasible.
- **Comparison with optimal solutions** (if available). Test the heuristics on small problems before scaling it up.
- **Comparison with Lower/Upper bounds** if already known (may be a target was already set)
- **Comparison with other metaheuristics methods** Reliability and stability over several runs Average gap between solutions while approaching the (local) optimum

Reasonable computational time Usually: between a few seconds upto a few hours

Main concepts to implement a Metaheuristic algorithm

- Representation of a solution
  - Vector of Binary values – 0/1 Knapsack, 0/1 IP problems
  - Vector of discrete values- Location , and assignment problems
  - Vector of continuous values on a real line – continuous, parameter optimization
  - Permutation – sequencing, scheduling, TSP
- Objective function

Main concepts to implement a Metaheuristic algorithm

- Constraint handling
  - **Reject strategies**- keep only feasible solution and reject infeasible ones automatically.
  - **Penalizing strategy** - infeasible solutions are considered and a penalty function is added to the objective function for including a infeasible solution during the search. s_t, s_{t+1}, s_{t+2} are the search sequence where s_{t+2} and s_t are feasible but s_{t+1} is infeasible and $f(s_{t+2})$ is better than $f(s_t)$. The penalized obj function is $f'(s) = f(s) + \lambda c(s)$ where $c(s)$ is the cost of infeasible s and λ is a weight.

Advantages	Disadvantages
Very flexible	Often need problem specific information / techniques
Often global optimizers	Optimality (convergence) may not be guaranteed
Often robust to problem size, problem instance and random variables	Lack of theoretic basis
May be only practical alternative	Different searches may yield different solutions to the same problem (stochastic)
	Stopping criteria
	Multiple search parameters

Algorithm Design & Analysis

Steps:

- Design the algorithm
- Encode the algorithm
  - Steps of algorithm
  - Data structures
- Apply the algorithm

Algorithm Efficiency

- Run-time
- Memory requirements
- Number of elementary computer operations to solve the problem in the worst case

Complexity analysis

- **Problem** is a collection of instances that share a mathematical form but differ in size and in the values of numerical constants in the problem form (i.e. generic model) **Example**: shortest path.
- **Instance** is a special case of a problem with specified data and parameters.
- An algorithm **solves** a problem if the algorithm is guaranteed to find the optimal solution for any instance.

Optimization vs Decision Problems

- **Optimization Problem** : A computational problem in which the object is to find the best of all possible solutions. (i.e. find a solution in the feasible region which has the minimum or maximum value of the objective function.)
- **Decision Problem**: A problem with a “yes” or “no” answer.
- What is the optimal value?
  - Is there a feasible solution to the problem with an objective function value equal to or superior to a specified threshold?

Complexity Classes of Problems

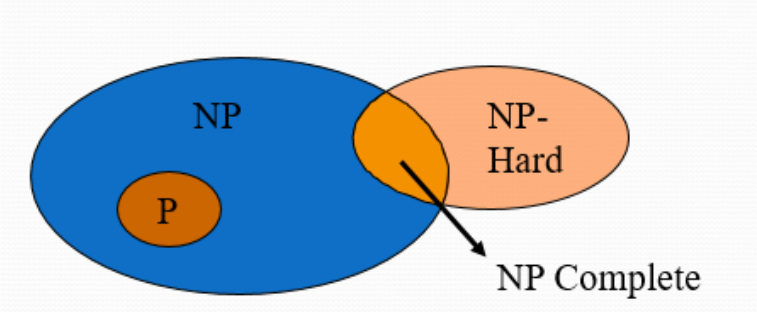
- **The class P:** Is the class of decision problems for which we can find a solution in polynomial time.
- Examples of P problems:
  - Shortest path
  - Minimum spanning tree
  - Network flow
  - Transportation, assignment and transshipment
  - Some single machine scheduling problems

Complexity Classes of Problems

- **NP = Nondeterministic Polynomial**
- **The class NP:** is the class of decision problems for which we can check solutions in polynomial time. i.e. easy to verify but not necessarily easy to solve
- A (decision) problem of class NP can be solved by a non deterministic machine in polynomial time.
- The class P is a subset of the class NP

NP Hard vs NP-Complete

- A problem P is said to be **NP-Hard** if all members of NP polynomially reduce to this problem.
- A problem P is said to be **NP-Complete** if
  - (a) $P \in NP$
  - (b) P is NP-Hard.



Combinatorial optimization problems

A combinatorial optimization problem $P = (S, f)$ can be defined by:

- Variables $X = x_1, x_2, ..., x_n$
- Variables domains $D_1, D_2, ..., D_n$
- Constraints among variables
- Objective function : $f : D_1 \times D_2 \times \times ... \times D_n \rightarrow \mathbb{R}^+$
- The set of all possible assignments solutions
  $S = \{s = \{(x_1, v_1), (x_2, v_2), ..., (x_n, v_n)\} \mid v_i \in D, s \text{ satifies all constraints}\}$

COP Example: The Assignment Problem

- n persons (i) and n tasks (j), it costs C_{ij} to let person i do task j, the objective is to find the minimal cost assignment

$$x_{ij} = \begin{cases} 1 & \text{If person } i \text{ does task } j \\ 0 & \text{Otherwise} \end{cases}$$

$$\max \sum_{i=1}^{n} \sum_{j=1}^{n} c_{ij} x_{ij}$$

$$\sum_{j=1}^{n} x_{ij} = 1 \qquad , i = 1, \ldots, n$$

$$\sum_{i=1}^{n} x_{ij} = 1 \qquad , j = 1, \ldots, n$$

COP Example: The Knapsack problem

- n items 1, ..., *n* available, weight a_i , profit c_i
- A selection shall be packed in a knapsack with capacity **b**. Objective: Find the selection of items that maximizes the profit

$$x_i = \begin{cases} 1 & \text{If the item i is in the knapsack} \\ 0 & \text{otherwise} \end{cases}$$

$$\max \sum_{i=1}^{n} c_i x_i \quad \text{s.t.}$$

$$\sum_{i=1}^{n} a_i x_i \leq b$$

Example: Knapsack Problem

- Knapsack with capacity 101
- 10 ”items”: 1,...,10
- Trivial solution: empty backpack, value 0
- Greedy solution, assign the items after value: (0000010000), value 85

Better suggestions?

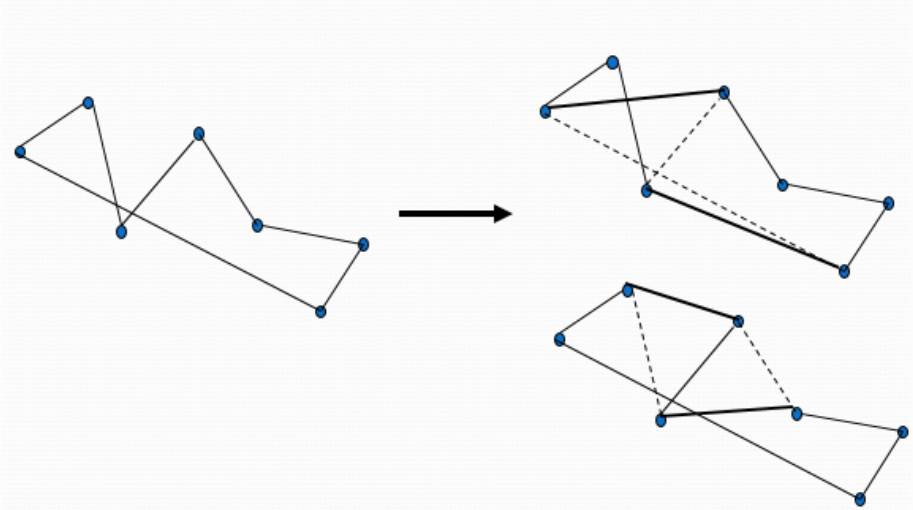
	1	2	3	4	5	6	7	8	9	10
Value	79	32	47	18	26	85	33	40	45	59
Size	85	26	48	21	22	95	43	45	55	52

Given a Solution: How to Find a Better One

- Modification of a given solution gives a "**neighbor** solution"
- A certain set of operations on a solution gives a set of **neighbor** solutions, a neighborhood
- Evaluations of neighbors:
  - Objective function value
  - Feasibility?

Example of neighborhood: TSP

- Operator: 2-opt
- How many neighbors?

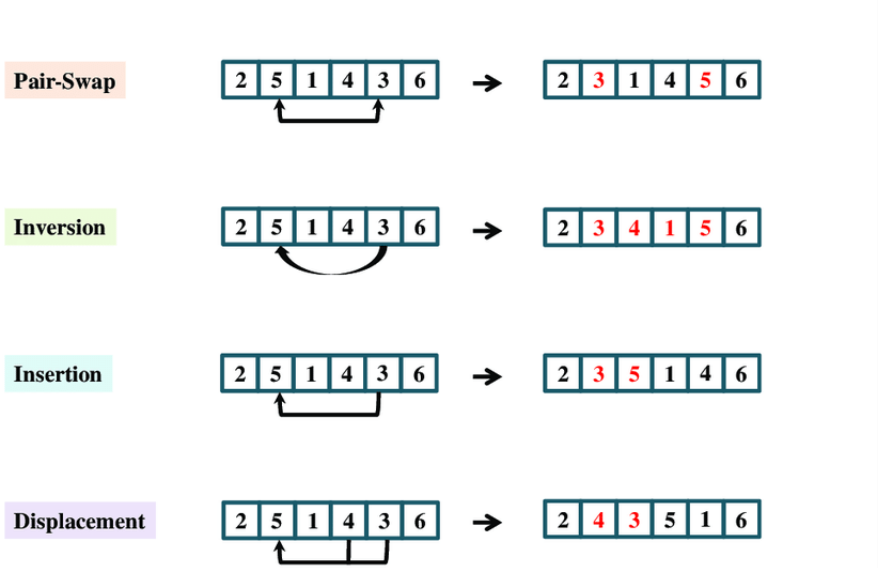


Example of neighborhood: Knapsack instance

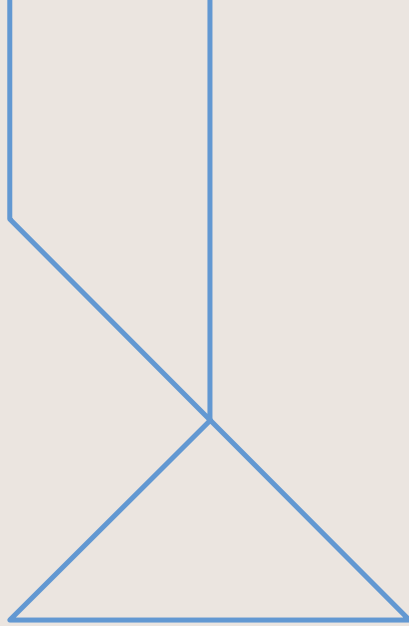
- Given solution **0010100000** value 73
- Natural operator: "**Flip**" a bit, i.e.:
  - If the item is in the knapsack, take it out
  - If the item is not in the knapsack, include it
- Some Neighbors:
  0110100000 value 105
  1010100000 value 152, **not feasible**
  0010000000 value 47

	1	2	3	4	5	6	7	8	9	10
Value	79	32	47	18	26	85	33	40	45	59
Size	85	26	48	21	22	95	43	45	55	52

Other Examples of neighborhood



ensia



- Trajectory methods -

Trajectory methods

Trajectory methods are so called because the search process designs a trajectory in the search space, starting from an initial state and dynamically adding a new better solution to the curve in each discrete time-step.
The characteristics of the trajectory provide information about the behavior of the algorithm and its effec tiveness with respect to the instance that is tackled. It is worth underlining that the dynamics is the result of the combina tion of algorithm, problem representation and problem instance.

Local Search

For combinatorial optimization problems

1. Start with initial configuration
2. Repeatedly search neighborhood (Successors) and select the best neighbor as candidate
3. Apply a cost function (or fitness function) and accept candidate if it is better than current
4. Stop if quality is sufficiently high, if no improvement can be found or after some fixed time
   - Candidate is always and only accepted if cost is lower than current configuration
   - Stop when no neighbor with lower cost (higher fitness) can be found

Implementation

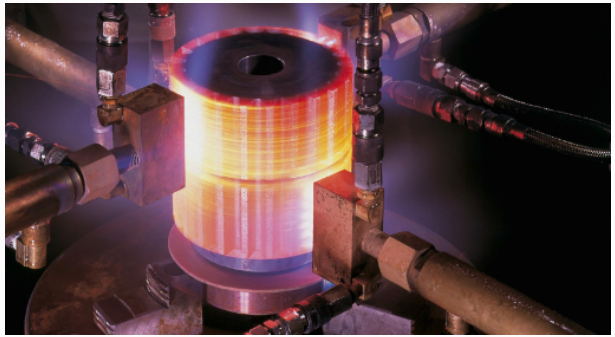
1. A method to generate initial configuration
2. A Successor function to generate new states
3. A Cost function
4. A Decision Criterion to select next candidates from the list of successors
5. A Stop Criterion

Improving local search

- Repeat algorithm many times with different initial configurations
- Use information gathered in previous runs
- Use a more complex Successor Function to jump out of local optimum
- Use a more complex Decision Criterion that sometimes could accept other alternative solutions (even worse than current)

Annealing in Metallurgy

Annealing is a heat treatment process that changes the physical and sometimes also the chemical properties of a material to increase ductility and reduce the hardness to make it more workable.
In metallurgy, annealing involves heating and then slowly cooling a material to alter its physical properties and reduce defects.



Simulated Annealing SA

In simulated annealing, the optimization problem is analogous to the material being annealed. The algorithm starts with an initial solution and iteratively explores the solution space. At each iteration, it randomly perturbs the current solution and evaluates whether the new solution is better or worse than the current one. If the new solution is better, it is accepted as the current solution. However, if the new solution is worse, it may still be accepted probabilistically, allowing the algorithm to escape local optima.

The acceptance probability for worse solutions decreases over time, analogous to the cooling process in metallurgy. Initially, the algorithm is allowed to accept worse solutions with a higher probability, but as the algorithm progresses, this probability decreases, allowing it to converge towards an optimal solution.

Simulated Annealing SA

- Metaheuristic search method based on local search methods
- Based upon the analogy with the simulation of the annealing of solids
- Do sometimes accept candidates with higher cost to escape from local optimum

Optimization problem	**Physical annealing**
Solution	Current state of the solid
Cost or Obj Value	Energy of current state
Control temperature T	Temperature
Optimal Solution	Minimum energy state
Local search	Quick cooling

SA general Algorithm

AlgorithmSA0

- T=upper limit
- Current= random initial state
- while (T does not reach lower limit)
  - New = random Successor from Current
    - if New improves Current
    - Current=New
    - else
    - Accept New with probability P
  - endif
  - cool(T)
- end while

Probability Function

- The probability of updating the state depends on the extent to which one solution is worse than the other:
- ΔE is the difference between cost values of the current and new states
- T: temperature of the system that is progressively decreasing

$$p = \exp\left(\frac{-\Delta E}{T}\right)$$

$$\Delta E = fcost(New) - fcost(Current)$$

AlgorithmSA

- Initialize T, T_{min}
- Current= random state
- while $(T > T_{min})$ & $(NoSolution)$
  - New = Random Successor from Current
  - $\Delta E = fcost(New) - fcost(Current)$

    - if ($\Delta E < 0$) (if candidate is better is accepted directly)
    - Current=New
    - else

      $p = e^{-\frac{\Delta E}{T}}$

      if p> $rand(0, 1)$ (accept solution randomly)

      Current=New

      endif
    - endif
    - cool(T)
- endwhile
- Return Current

Parameter Analysis

- If **T** is very **high**, convergence is very **slow**
- If **T** is very **low**, procedure will not converge
- T_{min} can be replaced by the maximum number of iterations
- The function $cool(T)$ decreases in each iteration
  - Depends upon the problem
  - Linear decreasing $T_i = \alpha T_{i-1}$ is the most common (α is the cooling coeffetient)
  - Other options: exponential decreasing, Cauchy, etc.

- SA is a general solution method that is easily applicable to a large number of problems
- ”Tuning” of the parameters (initial T, decrement of T, stop criterion) is relatively easy
- Generally the quality of the results of SA is good, although it can take a lot of time
- Results are generally not reproducible: another run can give a different result
- SA can leave an optimal solution and not find it again (so try to remember the best solution found so far)
- Proven to find the optimum under certain conditions; one of these conditions is that you must run forever

Tabu Search TS

Metaheuristic search method based on local search methods

Tabu states are states already visited or actions performed recently.

Tabu states are introduced to discourage the search from coming back to previously visited solutions.

Principle

In each iteration
- the best Successor (no Tabu) is selected
    - Current is included into the Tabu list for a certain number of iterations
- If a Successor is Tabu, then ”avoid” it and take the next one

Principle

- Runs until an external completion condition is satisfied
- The generated Successor could be worse than Current state
- Adaptive memory: a list of Tabu states is updated, the undesirable states
- Tenure: number of iterations that will keep a state on the Tabu list
- Some Successor, even being Tabu, might offer an exceptionally good solution.
- **The aspiration criterion** allows you to check whether a Tabu state is better than the best state, and if so, to select it, by updating the corresponding Tabu list.

TS algorithm

```
Algorithm Tabu_Search
    Current= random or initial state
    Best=Current
    Initial values for tabu list, tenure, iteration counter and max number of iterations
    While <stop conditions>
        Obtain the list of succesor states ordered by cost function
        While <successor list not empty and not a new current state is generated >
            New=Next(Sucessor list)
            if Cost(New) < Cost(Best) %% Tabu or not tabu but improves the best
                Current=Sucessor
                Best=Current
            elsif New is Not Tabu
                Current=Sucessor
            else
                Take next sucessor from the ordered list of sucessors
            endif
            Update tabu list (decreasing tenure and adding new tabu state)
        endWhile
        Update rest of variables
    endWhile
end
```

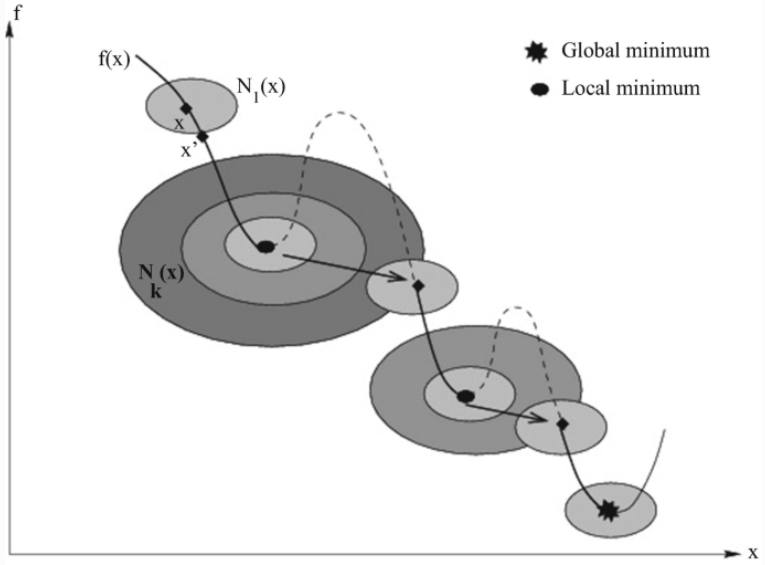
Variable Neighborhood Search VNS

Variable Neighborhood Search is a new and widely applicable metaheuristic that makes use of a strategy based on dynamically changing neighborhood structures during the search process. VNS provides a general framework and many variants exist for specific requirements. VNS doesn't follow a trajectory, but it searches for new solutions in increasingly distant neighborhoods of the current solution, jumping only if they are better than the current best solution.

Principle

1. Initialization: Start with an initial solution to the optimization problem.
2. Local Search within Neighborhoods: Perform a local search algorithm (such as hill climbing or simulated annealing) to optimize the current solution within its neighborhood.
3. Change Neighborhoods: Once a local optimum is reached within the current neighborhood, perturb the solution to move to a different neighborhood.
4. Repeat: Apply the local search algorithm within the new neighborhood to further optimize the solution.
5. Stopping Criterion: Repeat steps 3 and 4 until a stopping criterion is met, such as a predetermined number of iterations or reaching a certain level of optimization.

VNS



VNS Flowchart